

ITA0605 – MACHINE LEARNING

MATHIMOZHI.E.R

192321121

QUESTION -1:

$P_A = 0.25$

$P_B = 0.50$

$P_C = 0.25$

$P_{\text{Blue_given_A}} = 4 / 12$ # 4 blue out of 12

$P_{\text{Blue_given_B}} = 7 / 15$ # 7 blue out of 15

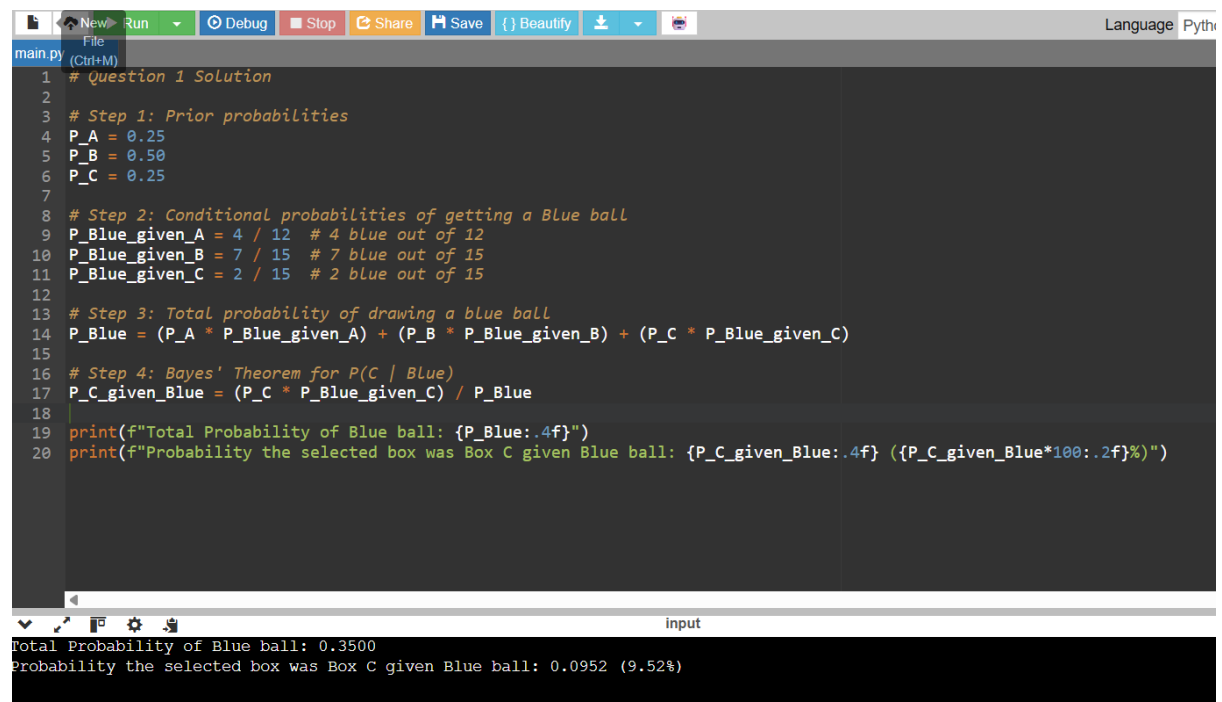
$P_{\text{Blue_given_C}} = 2 / 15$ # 2 blue out of 15

$P_{\text{Blue}} = (P_A * P_{\text{Blue_given_A}}) + (P_B * P_{\text{Blue_given_B}}) + (P_C * P_{\text{Blue_given_C}})$

$P_{\text{C_given_Blue}} = (P_C * P_{\text{Blue_given_C}}) / P_{\text{Blue}}$

print(f"Total Probability of Blue ball: {P_Blue:.4f}")

print(f"Probability the selected box was Box C given Blue ball: {P_C_given_Blue:.4f} ({P_C_given_Blue*100:.2f}%)")

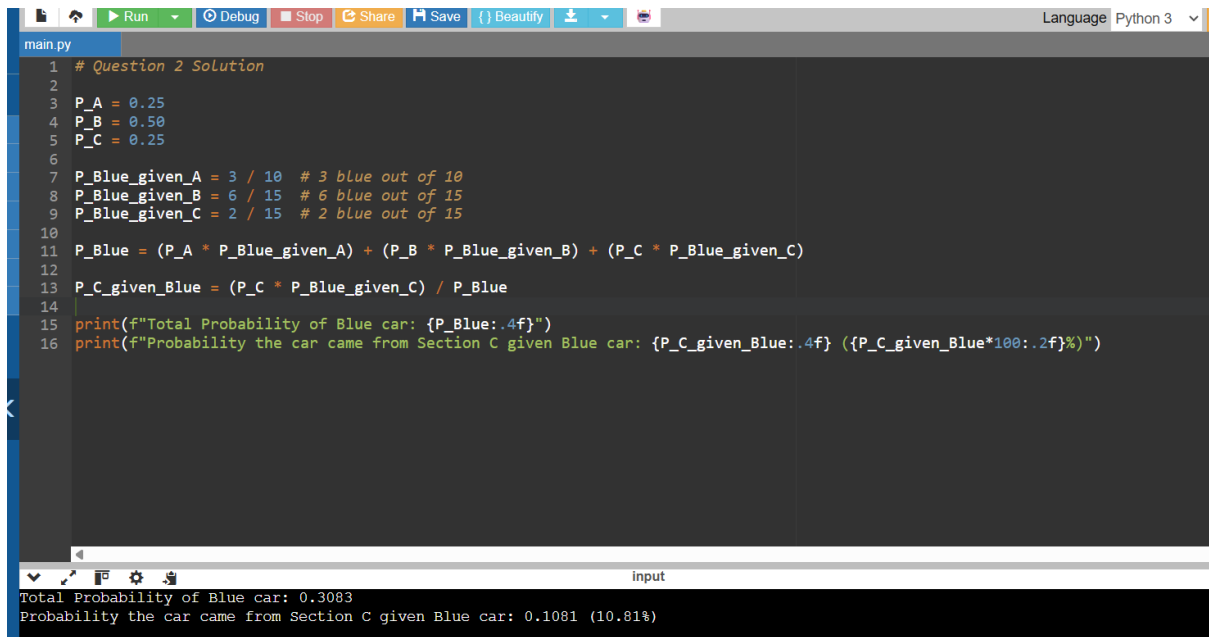


```
1 # Question 1 Solution
2
3 # Step 1: Prior probabilities
4 P_A = 0.25
5 P_B = 0.50
6 P_C = 0.25
7
8 # Step 2: Conditional probabilities of getting a Blue ball
9 P_Blue_given_A = 4 / 12 # 4 blue out of 12
10 P_Blue_given_B = 7 / 15 # 7 blue out of 15
11 P_Blue_given_C = 2 / 15 # 2 blue out of 15
12
13 # Step 3: Total probability of drawing a blue ball
14 P_Blue = (P_A * P_Blue_given_A) + (P_B * P_Blue_given_B) + (P_C * P_Blue_given_C)
15
16 # Step 4: Bayes' Theorem for P(C | Blue)
17 P_C_given_Blue = (P_C * P_Blue_given_C) / P_Blue
18
19 print(f"Total Probability of Blue ball: {P_Blue:.4f}")
20 print(f"Probability the selected box was Box C given Blue ball: {P_C_given_Blue:.4f} ({P_C_given_Blue*100:.2f}%)")
```

input

Total Probability of Blue ball: 0.3500
Probability the selected box was Box C given Blue ball: 0.0952 (9.52%)

QUESTION-2:

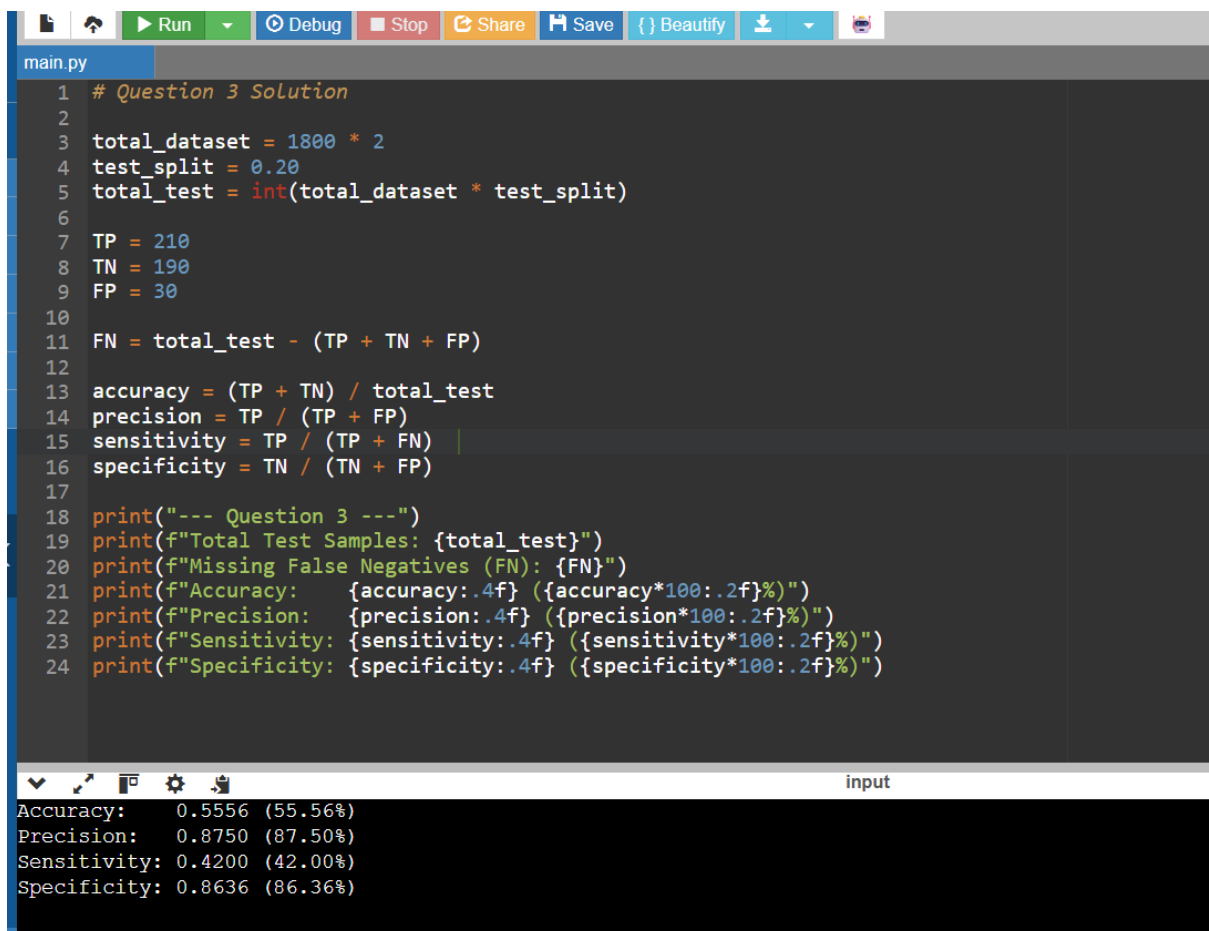


```
main.py
1 # Question 2 Solution
2
3 P_A = 0.25
4 P_B = 0.50
5 P_C = 0.25
6
7 P_Blue_given_A = 3 / 10 # 3 blue out of 10
8 P_Blue_given_B = 6 / 15 # 6 blue out of 15
9 P_Blue_given_C = 2 / 15 # 2 blue out of 15
10
11 P_Blue = (P_A * P_Blue_given_A) + (P_B * P_Blue_given_B) + (P_C * P_Blue_given_C)
12
13 P_C_given_Blue = (P_C * P_Blue_given_C) / P_Blue
14
15 print(f"Total Probability of Blue car: {P_Blue:.4f}")
16 print(f"Probability the car came from Section C given Blue car: {P_C_given_Blue:.4f} ({P_C_given_Blue*100:.2f}%)")
```

input

Total Probability of Blue car: 0.3083
Probability the car came from Section C given Blue car: 0.1081 (10.81%)

QUESTION-3



```
main.py
1 # Question 3 Solution
2
3 total_dataset = 1800 * 2
4 test_split = 0.20
5 total_test = int(total_dataset * test_split)
6
7 TP = 210
8 TN = 190
9 FP = 30
10
11 FN = total_test - (TP + TN + FP)
12
13 accuracy = (TP + TN) / total_test
14 precision = TP / (TP + FP)
15 sensitivity = TP / (TP + FN)
16 specificity = TN / (TN + FP)
17
18 print("--- Question 3 ---")
19 print(f"Total Test Samples: {total_test}")
20 print(f"Missing False Negatives (FN): {FN}")
21 print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
22 print(f"Precision: {precision:.4f} ({precision*100:.2f}%)")
23 print(f"Sensitivity: {sensitivity:.4f} ({sensitivity*100:.2f}%)")
24 print(f"Specificity: {specificity:.4f} ({specificity*100:.2f}%)")
```

input

Accuracy: 0.5556 (55.56%)
Precision: 0.8750 (87.50%)
Sensitivity: 0.4200 (42.00%)
Specificity: 0.8636 (86.36%)

QUESTION-4:

```
main.py
1 # Question 4 Solution
2
3 epochs = list(range(1, 9))
4 train_loss = [2.8, 2.5, 2.2, 2.0, 1.8, 1.6, 1.5, 1.3]
5 val_loss = [2.4, 2.2, 2.0, 2.1, 2.3, 2.6, 2.9, 3.2]
6
7 overfit_epoch = None
8 for i in range(1, len(val_loss)):
9     if val_loss[i] > val_loss[i-1]:
10         overfit_epoch = epochs[i]
11         break
12
13 print(f'i) Overfitting begins at Epoch: {overfit_epoch}")
14 print("ii) Techniques: 1. Early Stopping, 2. Dropout / L2 Regularization")
15 print("iii) Generalization Impact: Model memorizes noise, leading to poor test accuracy on unseen data.")
```

input

```
i) Overfitting begins at Epoch: 4
ii) Techniques: 1. Early Stopping, 2. Dropout / L2 Regularization
iii) Generalization Impact: Model memorizes noise, leading to poor test accuracy on unseen data.
```

QUESTION – 5:

```
1 # Question 5 Solution
2
3 epochs = list(range(1, 9))
4 train_acc = [60, 65, 70, 75, 80, 85, 88, 92]
5 val_acc = [58, 63, 68, 72, 73, 72, 70, 68]
6
7 # Overfitting begins right after peak validation accuracy
8 max_val_acc = max(val_acc)
9 peak_epoch = epochs[val_acc.index(max_val_acc)]
10 overfit_epoch = peak_epoch # Overfitting manifests as val_acc drops after epoch 5
11
12 print(f'i) Peak validation performance at Epoch {peak_epoch}. Overfitting starts at Epoch: {peak_epoch + 1} (drop in validation accuracy)')
13 print("ii) Cause: The model memorizes training noise rather than learning general patterns.")
14 print("iii) Solutions: 1. Data Augmentation, 2. Reduce model complexity / Add Dropout.")
```

input

```
i) Peak validation performance at Epoch 5. Overfitting starts at Epoch: 6 (drop in validation accuracy).
ii) Cause: The model memorizes training noise rather than learning general patterns.
iii) Solutions: 1. Data Augmentation, 2. Reduce model complexity / Add Dropout.
```

..Program finished with exit code 0
Press ENTER to exit console.